
Development of a Method for Protecting Mobile Payment Systems from Relay and Skimming Attacks

By

ZHUMAGAZY ABYLAY



School of Cybersecurity
ASTANA IT UNIVERSITY

6B06301 — Cybersecurity
Supervisor: PhD Amirova Akzhibek

JUNE 2026
ASTANA

Abstract

Mobile contactless payments based on Near Field Communication (NFC) technology have become a dominant payment paradigm worldwide, yet they remain vulnerable to two major classes of attack: *relay attacks*, where an adversary silently proxies a transaction over a long distance, and *skimming attacks*, where payment data are harvested from a card or phone without the owner’s knowledge or consent.

This diploma project presents the **HMFPV framework** (Hardware-ML-Freshness-Proximity-Verification) — a three-level, defence-in-depth security stack integrated directly into the Android Host-Based Card Emulation (HCE) subsystem. Level 1 uses the device’s linear accelerometer to enforce *intentional proximity*: a transaction is blocked unless the phone was physically moved shortly before the tap, preventing skimming of a stationary device. Level 2 issues time-stamped HMAC-SHA-256 signed session tokens whose keys never leave the Android Keystore hardware boundary, providing cryptographic anti-replay protection. Level 3 feeds the NFC round-trip time (RTT) and time-since-last-movement into a TensorFlow Lite Multi-Layer Perceptron (MLP) classifier that distinguishes normal transactions from relay and skimming anomalies entirely on-device and without cloud connectivity.

A synthetic dataset of 500 labelled transactions was engineered to reflect realistic RTT distributions for normal (<80 ms), relay (>80 ms), and skimming (>2000 ms of inactivity) scenarios. The MLP, with hidden layers of sizes (16, 8), achieved 100% accuracy on the held-out test set and was exported to the `.tflite` format for sub-millisecond inference. A fully functional Android prototype comprising a POS Terminal activity and an HCE card service was built in Kotlin and tested against all three attack scenarios. Experimental results confirm that the three-layer stack eliminates false negatives for both relay and skimming attacks while introducing only ~ 7 ms of processing overhead for legitimate payments.

Keywords: NFC security, relay attack, skimming attack, Host-Based Card Emulation, Android Keystore, HMAC-SHA-256, machine learning, TensorFlow Lite, mobile payments, defence-in-depth, accelerometer.

Аннотация

Мобильные бесконтактные платежи на основе технологии NFC (Near Field Communication) стали доминирующей платёжной парадигмой во всём мире, однако по-прежнему остаются уязвимыми к двум основным классам атак: *relay-атакам*, при которых злоумышленник незаметно ретранслирует транзакцию на большое расстояние, и *скиммингу*, при котором платёжные данные извлекаются с карты или смартфона без ведома и согласия владельца.

В данном дипломном проекте представлена **система НМFPV** (Hardware-ML-Freshness-Proximity-Verification) — трёхуровневый стек защиты по принципу «глубокой эшелонированной обороны», интегрированный непосредственно в подсистему Android Host-Based Card Emulation (HCE). Уровень 1 использует линейный акселерометр устройства для обеспечения *осознанной близости*: транзакция блокируется, если телефон не был физически перемещён незадолго до поднесения к терминалу, что исключает скимминг неподвижного устройства. Уровень 2 формирует сессионные токены с временной меткой и подписью HMAC-SHA-256, ключи которых никогда не покидают аппаратное хранилище Android Keystore, обеспечивая криптографическую защиту от повторного использования токенов. Уровень 3 подаёт время кругового обхода (RTT) NFC-команды и время с момента последнего движения устройства на классификатор TensorFlow Lite на основе многослойного персептрона (MLP), который разграничивает легитимные транзакции, relay-атаки и скимминг полностью на устройстве, без подключения к облаку.

Синтетический датасет из 500 размеченных транзакций был сформирован для отражения реалистичных распределений RTT: нормальные транзакции (<80 мс), relay-атаки (>80 мс) и скимминг (>2000 мс бездействия). Модель MLP со скрытыми слоями размером (16, 8) достигла точности 100% на отложенной тестовой выборке и была экспортирована в формат `.tflite` для инференса длительностью менее одной миллисекунды. Полностью функциональный прототип Android, включающий активность POS-терминала и HCE-сервис карты, написан на Kotlin и протестирован во всех трёх сценариях атак. Экспериментальные результаты подтверждают, что трёхуровневый стек полностью устраняет пропуски обнаружения как relay-атак, так и скимминга, добавляя к легитимным платежам лишь ~7 мс накладных расходов.

Ключевые слова: безопасность NFC, relay-атака, скимминг, эмуляция карты на основе хоста, Android Keystore, HMAC-SHA-256, машинное обучение, TensorFlow

Lite, мобильные платежи, эшелонированная оборона, акселерометр.

Аңдатпа

NFC (Near Field Communication) технологиясына негізделген мобильді байланыссыз төлемдер бүкіл әлемде үстем төлем парадигмасына айналды, алайда олар екі негізгі шабуыл түріне осал болып қалуда: *relay-шабуылдар* — мұнда қарсылас транзакцияны үлкен қашықтыққа байқатпай ретрансляциялайды, және *скимминг* — бұл жағдайда төлем деректері иесінің рұқсатынсыз карта немесе смартфоннан алынады.

Бұл диплом жобасында **НМFRPV жүйесі** (Hardware-ML-Freshness-Proximity-Verification) — Android Host-Based Card Emulation (HCE) ішкі жүйесіне тікелей енгізілген үш деңгейлі, тереңдетілген қорғаныс стегі ұсынылады. 1-деңгей *саналы жақындықты* қамтамасыз ету үшін құрылғының сызықтық акселерометрін пайдаланады: телефон терминалға ұсынылардан бұрын физикалық түрде жылжытылмаса, транзакция бұғатталады, бұл қозғалыссыз құрылғының скиммингіне жол бермейді. 2-деңгей уақыт белгісі бар және HMAC-SHA-256 қолтаңбасы бар сессиялық токендерді шығарады; олардың кілттері ешқашан Android Keystore аппараттық шекарасынан шықпайды, бұл токендерді қайта пайдаланудан криптографиялық қорғауды қамтамасыз етеді. 3-деңгей NFC пәрменінің кідіріс уақытын (RTT) және соңғы қозғалыстан өткен уақытты TensorFlow Lite негізіндегі көп қабатты перцептрон (MLP) классификаторына береді; ол қалыпты транзакцияларды, relay-шабуылдарды және скиммингті тек құрылғыда, бұлтқа қосылмай ажыратады.

500 таңбаланған транзакциядан тұратын синтетикалық деректер жиыны RTT-ның реалистік таралуларын бейнелеу үшін жасалды: қалыпты транзакциялар (<80 мс), relay-шабуылдар (>80 мс) және скимминг (>2000 мс белсенділіксіздік). (16, 8) өлшемді жасырын қабаттары бар MLP моделі ұсталып қалған тест жиынтығында 100% дәлдікке жетті және бір миллисекундтан аз уақытта инференс жасау үшін `.tflite` форматына экспортталды. POS-терминал активтілігін және HCE-карта қызметін қамтитын толық функционалды Android прототипі Kotlin тілінде жазылып, үш шабуыл сценарийінің барлығына қарсы сыналды. Эксперименттік нәтижелер үш деңгейлі стектің relay-шабуылдар мен скимминг үшін де жалған теріс нәтижелерді толықтай жоятынын және заңды төлемдерге тек ~7 мс қосымша кідіріс қосатынын растайды.

Түйінді сөздер: NFC қауіпсіздігі, relay-шабуыл, скимминг, хостқа негізделген карта эмуляциясы, Android Keystore, HMAC-SHA-256, машиналық оқыту, TensorFlow Lite, мобильді төлемдер, тереңдетілген қорғаныс, акселерометр.

Dedication and Acknowledgements

I dedicate this work to my family, whose unwavering support made this journey possible.

I am sincerely grateful to my supervisor, PhD Amirova Akzhibek, for her expert guidance, constructive feedback, and continuous encouragement throughout this project.

I also thank the faculty of the School of Computer Science and Cybersecurity at Astana IT University for providing an inspiring academic environment and access to research resources.

Declaration of Academic Integrity

I, **Zhumagazy Abylay**, student of Astana IT University, group CS-2304, hereby declare that this diploma project, entitled

*Development of a Method for Protecting Mobile Payment Systems
from Relay and Skimming Attacks*

is my own original work carried out under the supervision of **PhD Amirova Akzhibek**. I further declare that:

1. All sources of information used in this work are cited and referenced in accordance with the bibliographic requirements of Astana IT University (document DP-AITU-19).
2. All external software libraries, frameworks, and tools used in the implementation (including Android SDK, TensorFlow Lite, scikit-learn, and the Kotlin standard library) are properly acknowledged in the Technology Stack section of Chapter 2 and in the bibliography.
3. The Android application prototype, including `SecurePaymentService.kt`, `MainActivity.kt`, `KeystoreHelper.kt`, `TransactionLogger.kt`, and `StatsActivity.kt`, was written by me independently. Any code segments adapted from official Android documentation or open-source examples are identified in the source code comments.
4. The synthetic dataset used to train the MLP classifier was generated by me using Python and scikit-learn. No pre-existing labelled NFC dataset was used without acknowledgement.
5. This work has not been submitted, in whole or in part, for any other academic qualification or assessment at this or any other institution.

6. I understand that violation of the Academic Integrity Policy of Astana IT University (document DP-AITU-05) may result in disqualification of this work.

Student signature: _____

Full name: Zhumagazy Abylay

Group: CS-2304

Date: _____

Supervisor signature: _____

Full name: PhD Amirova Akzhibek

Date: _____

Definitions

Following terms are used in this work:

NFC (Near Field Communication)	A short-range wireless communication standard operating at 13.56 MHz over distances typically not exceeding 10 cm, used for contactless payments, access control, and data exchange.
Relay Attack	An attack in which two colluding adversaries use a wired or wireless channel to transparently proxy NFC messages between a victim card/phone and a legitimate reader, extending the effective communication range without the knowledge of either endpoint.
Skimming Attack	An attack in which a rogue NFC reader covertly interrogates a contactless card or NFC-enabled phone to harvest payment credentials without the cardholder's awareness or consent.
HCE (Host-Based Card Emulation)	An Android architecture that allows an application running on the device CPU to act as a contactless smart card, routing APDU commands from an NFC reader to application logic rather than a secure element.
APDU (Application Protocol Data Unit)	The communication unit between a smart-card application and an NFC reader, defined by ISO/IEC 7816.
HMAC (Hash-Based Message Authentication Code)	A mechanism for verifying both the integrity and authenticity of a message by combining a cryptographic hash function with a shared secret key.

Android Keystore	A system service that stores cryptographic keys in hardware-backed secure storage, preventing key extraction even with root access.
RTT (Round-Trip Time)	The elapsed time between sending an NFC command and receiving the corresponding response; used as a distance-bounding proxy in the HMFPV framework.
MLP (Multi-Layer Perceptron)	A class of feedforward artificial neural network comprising one or more hidden layers between the input and output layers, used here as the on-device anomaly classifier.
TFLite (TensorFlow Lite)	A lightweight inference runtime from Google for deploying machine-learning models on mobile and embedded devices without cloud dependency.
EMV	A global standard for chip-based payment cards and terminals, named after Europay, Mastercard, and Visa, which governs cryptographic authentication during payment transactions.

Designations and Abbreviations

Following designations and abbreviations are used in this work:

NFC	Near Field Communication
HCE	Host-Based Card Emulation
APDU	Application Protocol Data Unit
HMAC	Hash-Based Message Authentication Code
SHA-256	Secure Hash Algorithm 256-bit
RTT	Round-Trip Time
MLP	Multi-Layer Perceptron
ML	Machine Learning
AI	Artificial Intelligence
TFLite	TensorFlow Lite
EMV	Europay, Mastercard, Visa (payment standard)
ISO	International Organization for Standardization
HMFPV	Hardware-ML-Freshness-Proximity-Verification (the proposed framework)
POS	Point of Sale
SDK	Software Development Kit
API	Application Programming Interface
JVM	Java Virtual Machine
AES	Advanced Encryption Standard
TEE	Trusted Execution Environment

List of Tables

TABLE	Page
1.1 Command APDU (C-APDU) field structure per ISO/IEC 7816-4	7
1.2 Structured comparison of NFC payment security solutions	9
1.3 Summary of reviewed works by research focus	10
2.1 Dataset generation parameters per class	19
2.2 Synthetic dataset class distribution	20
2.3 Confusion matrix of the MLP classifier on the test set	21
2.4 Technology stack	23
2.5 Threat-to-layer mapping in the HMFPV framework	25
3.1 Comparative security analysis: traditional POS vs. HMFPV AI-Terminal . .	32
3.2 Per-layer processing overhead (measured on Xiaomi Redmi Note 11)	32
3.3 Edge-case and boundary-condition test results	34
3.4 MLP training metrics at selected epochs	35

List of Figures

FIGURE	Page
2.1 HMFPV framework architecture overview	14
2.2 HMFPV Transactions Classification Map	20
2.3 Message sequence diagram for a successful legitimate HMFPV transaction	24
3.1 Normal payment: all three layers pass (Sensor OK, Keystore OK, ML OK). The terminal displays <i>SAFE PAYMENT</i> , ML verdict: Normal, RTT = 57.3 ms.	28
3.2 Skimming attack blocked at Level 1: the accelerometer gate detected no move- ment. The terminal displays <i>SKIMMING ATTACK</i> , Sensor: FAIL. Keystore and ML layers were not invoked.	29
3.3 Replay/relay attack blocked at Level 2: the HMAC token was expired by 31 753 ms, far exceeding the 5 000 ms freshness window. Sensor and Keystore passed; ML was not reached.	30
3.4 Transaction statistics screen (<i>StatsActivity</i>): 9 total transactions, 8 safe (88%), 0 relay (0%), 1 skimming (11%). Average RTT = 77.9 ms. The log confirms the experimental results of Section 3.2.	31

Table of Contents

Abstract	i
Аннотация	ii
Аңдатпа	iv
Dedication and Acknowledgements	v
Declaration of Academic Integrity	vi
Definitions	viii
Designations and Abbreviations	x
List of Tables	xi
List of Figures	xii
Introduction	xvi
1 Literature Review	1
1.1 NFC Payment Architecture and the Trust Model	1
1.2 Relay Attacks	1
1.3 Skimming Attacks	3
1.4 Machine Learning for NFC Attack Detection	3
1.5 Threat Model	4
1.5.1 Adversary Capabilities	4
1.5.2 Protected Assets	5
1.5.3 Attack Classes in Scope	5

1.5.4	Security Goals	6
1.6	APDU Command Structure and Attack Vectors	6
1.6.1	Command APDU Structure	6
1.6.2	AID Selection Sequence	7
1.6.3	Relay Attack at the APDU Level	8
1.6.4	Skimming Attack at the APDU Level	8
1.7	Structured Comparison of Related Works	8
1.8	Summary and Research Gap	9
2	Methodology and Implementation	13
2.1	The HMFPV Framework: Design Principles	13
2.2	Level 1: Physical Accelerometer Gate	14
2.2.1	Threat Model	14
2.2.2	Implementation	14
2.3	Level 2: HMAC-SHA-256 Dynamic Token with Android Keystore	16
2.3.1	Threat Model	16
2.3.2	Key Management via Android Keystore	16
2.3.3	Token Construction and Verification	16
2.4	Level 3: On-Device MLP Anomaly Classifier	17
2.4.1	Dataset Engineering	17
2.4.2	Mathematical Description of Dataset Generation	18
2.4.3	Model Architecture and Training	20
2.4.4	TensorFlow Lite Deployment	21
2.5	Transaction Flow and Security Decision Logic	22
2.6	Transaction Logging and Statistics Module	23
2.7	Technology Stack	23
2.8	Transaction Sequence Diagram	23
2.9	Threat-to-Layer Mapping	25
3	Results and Discussion	26
3.1	Experimental Setup	26
3.2	Test Results by Scenario	27
3.2.1	Normal Payment	27
3.2.2	Relay Attack Simulation	27
3.2.3	Skimming Simulation	27
3.3	Prototype Screenshots	27

3.4	Comparative Security Analysis	31
3.5	Performance Analysis	32
3.6	Edge-Case and Boundary Condition Testing	33
3.6.1	High-Motion False-Positive Test	33
3.6.2	Electromagnetic Interference Test (Metro Environment)	33
3.6.3	Rapid Re-Tap Test	34
3.6.4	Summary of Edge-Case Results	34
3.7	MLP Training Convergence Analysis	35
3.7.1	Training and Validation Loss	35
3.7.2	Convergence Observations	35
3.8	Limitations and Future Work	36
Conclusion		38
Bibliography		40
Appendix A: Source Code Listings		43
	SecurePaymentService.kt	43
	KeystoreHelper.kt	45
	TransactionLogger.kt	46

Introduction

Contactless payment technology based on Near Field Communication (NFC) has undergone remarkable adoption over the past decade. According to industry reports, global contactless payment transaction volume surpassed \$10 trillion in 2023 and is projected to grow further as smartphone penetration rises in developing markets [1]. Kazakhstan has followed this trend, with the national payment system integrating NFC-based tokens issued by major card networks through mobile wallets on both Android and iOS platforms.

Despite its convenience, the NFC contactless payment model rests on a security assumption that is increasingly contested: the assumption that physical proximity implies authorisation. Two attack families directly exploit this assumption. *Relay attacks* allow a pair of cooperating adversaries to tunnel an NFC transaction over an arbitrary distance, making a card in a pocket appear to be tapping a reader in another country [2]. *Skimming attacks* go further, harvesting credentials from a card or device without any knowledge of the owner [3]. Classical EMV defences — static cryptograms, application transaction counters — were designed for chip-and-PIN scenarios and provide insufficient protection against relay adversaries who transparently forward all bytes in real time [4].

The academic literature has proposed several countermeasure categories: distance-bounding protocols that use ultra-tight timing constraints [5], ambient-sensor proximity detection that correlates physical context between phone and terminal [6], hardware-assisted ranging via Ultra-Wideband (UWB) [7], and machine-learning classifiers trained on transaction timing features [8, 9]. Each approach addresses a subset of the threat model, but few works integrate multiple independent layers into a single deployable prototype that can run offline on a commodity Android device.

Motivation. The central motivation for this project is the gap between academically validated countermeasures and practical, deployable Android applications. Host-Based Card Emulation (HCE), available since Android 4.4, allows software to emulate an NFC payment card without a dedicated secure element, lowering the barrier to implementation. This project exploits HCE to implement a three-level security stack that combines physical-sensor gating, cryptographic freshness verification, and on-device neural-network anomaly detection into a unified architecture called **HMFPV** (Hardware-ML-Freshness-Proximity-Verification).

Research objectives. The specific objectives of this diploma project are:

1. To survey and classify existing countermeasures against relay and skimming attacks on NFC payment systems.
2. To design a multi-layer defence architecture that independently blocks each attack class at the hardware, cryptographic, and inference levels.
3. To implement the architecture as a fully functional Android prototype using Kotlin, the Android Keystore API, and TensorFlow Lite.
4. To train and evaluate an MLP classifier on a purpose-built synthetic dataset that models realistic RTT and motion-context distributions.
5. To validate the prototype experimentally against all three threat scenarios (normal, relay, skimming).

Structure of the work. Chapter 1 reviews the relevant literature on NFC attack vectors and existing countermeasures. Chapter 2 presents the HMFPV framework design, the synthetic dataset, the ML model, and the implementation details of each security layer. Chapter 3 discusses experimental results, comparative analysis against traditional POS systems, and limitations. The conclusion summarises contributions and outlines directions for future work.

Chapter 1

Literature Review

1.1 NFC Payment Architecture and the Trust Model

The ISO/IEC 18092 standard governs Near Field Communication, a radio technology that operates at a carrier frequency of 13.56 MHz and whose practical communication range is constrained to roughly 10 cm under normal conditions. Within the Android platform, two architectural paths exist for contactless payment applications. The first relies on a dedicated hardware Secure Element (SE) — a tamper-resistant chip separate from the main processor. The second, Host-Based Card Emulation (HCE), removes the SE dependency entirely: NFC Application Protocol Data Units (APDUs) arriving from an external reader are delivered directly to application-layer software executing in the Android runtime. Google introduced HCE support at API level 19 (Android 4.4), and the majority of software payment wallets deployed today follow this architecture.

Both architectures share a common security assumption that has proven increasingly fragile: the belief that the physical constraints of the NFC radio channel are sufficient to guarantee that the device and the reader are in close proximity, and that this proximity can be equated with the cardholder’s intent to pay. It is precisely this assumption that relay and skimming adversaries target, as the following sections detail.

1.2 Relay Attacks

The relay attack concept in contactless-card settings was formally introduced by Hancke and Kuhn and later analysed in the HCE context by Hasan [10]. The attack requires two

cooperating devices placed at geographically separate locations. One device, positioned near the victim, masquerades as a legitimate payment terminal and silently captures all NFC traffic directed at the victim’s phone or card. A second device, stationed near a genuine POS terminal, impersonates a contactless card and feeds the intercepted bytes — in real time — to the real reader. Since the EMV protocol transmits cryptographic material without binding it to the physical channel, every byte forwarded by the relay pair arrives at the terminal indistinguishable from a direct tap. The transaction completes successfully from the terminal’s perspective, while the cardholder remains unaware that their device was used.

Bocek et al. [2] demonstrated a practical relay attack using off-the-shelf Android phones, achieving a round-trip overhead of approximately 300–600 ms. Dang et al. [3] extended relay attacks to cyber-physical systems, showing that NFC-enabled infrastructure such as door locks and transport gates is equally vulnerable.

The most principled defence against relay attacks is the family of *distance-bounding protocols* [5], which interleave rapid challenge-response rounds and bound the device’s distance by the speed of light. Boureanu et al. [4] conducted a security analysis of relay-resistant contactless payments based on formal models, showing that standard EMV does not satisfy distance-bounding properties. Shah and Sampalli [11] proposed an LFSR-based distance-bounding protocol for EMV that reduces computational overhead while preserving correctness.

A complementary approach is *ambient-sensor* proximity detection, in which the phone and terminal independently measure a shared physical observable (e.g., ambient light, sound, or vibration) and compare readings. Shepherd et al. [12] evaluated several ambient sensor channels for NFC proximity detection, finding that ambient light and audio correlate well under cooperative conditions but degrade in adversarial settings. Gurulian et al. [13] tested the approach with real relay attack data and concluded that ambient sensing alone provides insufficient security guarantees. Markantonakis et al. [6] reproduced this finding in a 2024 study and called for multi-channel, fused approaches.

Ultra-Wideband (UWB) radio, standardised in IEEE 802.15.4a, supports centimetre-level ranging. Coppola et al. [7] presented PURE, a payment protocol that combines NFC data exchange with UWB ranging to enforce proximity at the hardware level, achieving relay rejection with sub-centimetre accuracy. The limitation of UWB is that both the phone and the terminal must support the UWB radio, which restricts deployment to

newer hardware.

RTT-based detection offers a software-only alternative: relay forwarding necessarily adds latency, so a terminal that measures the time from sending a command to receiving a response can reject transactions whose RTT exceeds a threshold derived from the expected propagation delay over 10 cm. Gurulian [14] studied time-constrained mobile contactless transactions and found that commodity NFC hardware introduces jitter that complicates threshold setting. Li et al. [15] proposed a countermeasure based on RTT measurement in the NFC payment stack.

1.3 Skimming Attacks

Where relay attacks require two cooperating adversaries operating in real time, skimming is a simpler and more passive threat: an attacker equipped with a concealed NFC reader brings it within radio range of the victim’s device and issues standard APDU commands, harvesting whatever credential material the HCE service returns. No network infrastructure is needed, and the entire interaction may complete in under 100 ms without the cardholder noticing anything. Hasan [10] demonstrated that HCE services which respond to arbitrary AID selection commands are fully exposed to this form of credential harvesting.

The simplest hardware-level defence is to require the device to be in motion — detected via an accelerometer — before the HCE service will respond. Sako et al. [16] described a physical authentication scheme based on the Ascon cipher that requires a proximity gesture; their work is closest in spirit to the Level 1 layer of the HMFPV framework. Cavdar et al. [17] proposed a context-aware security model that considers device orientation, velocity, and ambient light simultaneously.

1.4 Machine Learning for NFC Attack Detection

Applying machine learning to NFC transaction security is a comparatively young research direction, but one that has gained momentum as on-device inference has become practical. Tagle [8] demonstrated that a real-time classifier fed with timing and metadata features extracted from NFC card transactions can achieve high fraud-detection accuracy without requiring backend infrastructure. A different angle was taken by Wang et al. [9], who showed that the physical-layer radio characteristics of a relayed NFC signal differ

measurably from a direct one, enabling deep-learning-based RF fingerprinting to separate legitimate from relayed traffic. Aljahdali et al. [18] trained a deep neural network specifically for wireless relay detection and reported over 98% accuracy on a balanced evaluation set.

Earlier work by Isnan Imran et al. [19] modelled NFC payment sequences as Markov Chains, detecting statistical deviations that are characteristic of relay-injected anomalies. Chakka and Saheb [20] conducted a systematic review of ML approaches for mobile payment fraud and identified round-trip timing and motion context as the feature classes with the highest discriminative power — a finding that directly motivates the HMFPV Level 3 feature design. Reddy et al. [21] took a hybrid approach, embedding an AI-based validation module into the EMV transaction flow as a server-side component; the HMFPV framework extends this idea by relocating inference entirely onto the device.

1.5 Threat Model

This section formally defines the adversary model and the assets to be protected, following the convention established by Avoine et al. [5] for distance-bounding security analyses.

1.5.1 Adversary Capabilities

The adversary $\mathcal{A}$ is assumed to be computationally bounded and unable to break standard cryptographic primitives (AES-256, HMAC-SHA-256) within the transaction validity window. Within these bounds, $\mathcal{A}$ may:

1. Observe, record, and replay all bytes transmitted over the NFC ISO-DEP channel.
2. Introduce arbitrary delays into the NFC communication channel by routing APDU bytes through a relay device and a wide-area network.
3. Deploy one or more concealed NFC readers in the physical vicinity of the victim's device without the victim's knowledge.
4. Attempt to submit forged or expired tokens to the terminal.
5. Physically shake the victim's device to produce a false accelerometer reading.

$\mathcal{A}$ is **not** assumed to have physical access to the device’s internals, root privileges on the device, or the ability to extract keys from the Android Trusted Execution Environment (TEE).

1.5.2 Protected Assets

The following assets must be protected:

- **Transaction authorisation:** the terminal’s decision to approve a payment must reflect a genuine, proximate, and intentional tap by the legitimate device owner.
- **HMAC signing key:** the symmetric key used to authenticate session tokens must never be exposed in plaintext outside the Android Keystore hardware boundary.
- **Token uniqueness:** each transaction token must be valid for only one use and within a short time window, preventing replay.

1.5.3 Attack Classes in Scope

Two attack families are in scope for this work:

Relay attack. $\mathcal{A}$ deploys a *ghost* device near the POS terminal and a *leech* device near the victim’s phone. APDU bytes are forwarded over a high-bandwidth channel, adding a measurable latency overhead Δ_{relay} to the round-trip time. Empirical measurements by Bocek et al. [2] and Gurulian et al. [13] place Δ_{relay} in the range 100–600 ms for WiFi-based relays.

Skimming attack. $\mathcal{A}$ brings a concealed NFC reader within range of the victim’s stationary device. The device responds to APDU commands without the owner’s knowledge because the HCE service does not require explicit user interaction before issuing a response.

Both attack classes exploit the same root vulnerability: the EMV contactless protocol’s implicit assumption that NFC’s limited range (<10 cm) guarantees physical proximity and user consent [4].

1.5.4 Security Goals

The HMFPV framework must satisfy the following security goals:

G1 – Relay resistance:

A transaction completed via a relay channel that introduces $\Delta_{\text{relay}} \geq 80$ ms additional RTT must be rejected with probability ≥ 0.99 .

G2 – Skimming resistance:

A transaction initiated against a device that has not been physically moved for ≥ 2000 ms must be blocked before any token material is transmitted.

G3 – Replay resistance:

A token captured from a legitimate transaction must be rejected if presented more than 5000 ms after its creation timestamp.

G4 – Key non-extractability:

The HMAC signing key must be stored exclusively within the Android Keystore hardware boundary and must not appear in application memory, logs, or backup storage in plaintext form.

Goals G1, G2, G3, and G4 correspond to Levels 3, 1, 2, and 2 of the HMFPV framework, respectively, as detailed in Chapter 2.

1.6 APDU Command Structure and Attack Vectors

Application Protocol Data Units (APDUs) are the fundamental communication units between an NFC reader and a card emulator, defined by the ISO/IEC 7816-4 standard. Understanding the exact structure of APDU commands is essential for analysing how relay and skimming attacks operate at the protocol level.

1.6.1 Command APDU Structure

A command APDU (C-APDU) has the following format:

Table 1.1: Command APDU (C-APDU) field structure per ISO/IEC 7816-4

Field	Length	Value	Description
CLA	1 byte	0x00–0xFF	Class byte: identifies the command class (e.g., 0x00 for ISO standard, 0x80 for proprietary).
INS	1 byte	0x00–0xFF	Instruction byte: specifies the command (e.g., 0xA4 for SELECT, 0xB0 for READ BINARY).
P1	1 byte	0x00–0xFF	Parameter 1: first command parameter (semantics depend on INS).
P2	1 byte	0x00–0xFF	Parameter 2: second command parameter.
Lc	0 or 3 bytes	0x00–0xFFFF	Length of command data field; absent if no data is sent.
Data	L_c bytes	variable	Command data (e.g., AID in a SELECT command, PIN block in authentication).
Le	0, 1, or 3 bytes	0x00–0xFFFF	Expected length of the response; 0x00 means “any length”.

A response APDU (R-APDU) consists of an optional data field followed by a mandatory two-byte status word (SW1, SW2). The status word 0x9000 indicates success; 0x6A80 indicates “Wrong Data” and is used by the HMFPV Level 1 gate to signal a skimming block.

1.6.2 AID Selection Sequence

Every contactless EMV transaction begins with a SELECT command that identifies the payment application by its Application Identifier (AID). The standard SELECT C-APDU for a Visa payment application has the following structure:

00 A4 04 00 07 A0 00 00 00 03 10 10 00

Here: CLA=00, INS=A4 (SELECT FILE), P1=04 (select by AID), P2=00, Lc=07 (AID length), Data=A0 00 00 00 03 10 10 (Visa AID), Le=00.

In the HMFPV prototype the POS terminal side (`MainActivity`) broadcasts this SELECT command to trigger the HCE service. On receiving the command, `SecurePaymentService` executes the three-level check before returning any response.

1.6.3 Relay Attack at the APDU Level

During a relay attack, the adversary’s *mole* device intercepts the SELECT C-APDU and forwards its raw bytes over a TCP/IP channel to the *ghost* device, which replays them to the victim phone. Because EMV does not authenticate the transport channel, the phone cannot distinguish a local terminal from a proxied one. The only observable artefact is the added round-trip latency Δ_{relay} , which the HMFPV Level 3 MLP classifier detects via the RTT feature.

Empirically, a Wi-Fi relay introduces $\Delta_{\text{relay}} \approx 100\text{--}600$ ms [2, 13]. A 4G mobile relay can reduce this to $\sim 80\text{--}150$ ms, which still exceeds the normal NFC RTT of 10–40 ms and remains detectable at the 80 ms decision boundary.

1.6.4 Skimming Attack at the APDU Level

A skimming attack begins with the adversary’s rogue reader broadcasting a SELECT C-APDU to any nearby HCE-enabled phone. Without the HMFPV Level 1 gate, the phone’s `SecurePaymentService` would respond with a valid token, leaking HMAC material to the attacker. The Level 1 accelerometer guard prevents this by checking $\Delta t_{\text{move}} \leq 2000$ ms before any token is generated; if the device has been stationary, the service returns 0x6A80 without disclosing any cryptographic material.

1.7 Structured Comparison of Related Works

Table 1.2 consolidates the literature reviewed in the preceding sections into a single structured comparison. Works are evaluated against six criteria relevant to the threat model defined in Section 1.5: relay attack detection, skimming protection, machine-learning-based classification, hardware-backed key storage, compatibility with commodity Android HCE, and absence of specialised hardware requirements.

Table 1.2: Structured comparison of NFC payment security solutions

Solution		Relay De- tect.	Skimming Prot.	ML	HW Key- store	HCE Com- pat.	No Spec. HW
EMV (Visa/MC)	3DS	Partial	No	No	Yes	No	Yes
Distance-Bounding [5]		Yes	No	No	No	No	Yes
Ambient Sensor [12, 13]		Yes	Partial	No	No	No	Yes
PURE / UWB [7]		Yes	No	No	No	No	No
RF Fingerprint [9]		Yes	No	Yes	No	No	Yes
ML Fraud Detect. [8]		Yes	No	Yes	No	No	Yes
Deep Learning [18]		Yes	No	Yes	No	No	Yes
Context-Aware [17]		Yes	Partial	No	No	Yes	Yes
Enhanced EMV [21]		Yes	No	Yes	No	No	Yes
HMFPV (this work)		Yes	Yes	Yes	Yes	Yes	Yes

The comparison reveals that the HMFPV framework is the only solution satisfying all six criteria simultaneously. In particular, no prior work combines skimming protection via accelerometer with hardware-backed Keystore cryptography and on-device ML classification in a single Android HCE application that requires no specialised hardware beyond a standard NFC-equipped smartphone.

1.8 Summary and Research Gap

Table 1.3 summarises the 32 reviewed works by research focus. Reading across the literature, four recurring patterns stand out:

1. **Distance-bounding is theoretically sound but hardware-constrained.** Protocols that enforce proximity by bounding the speed-of-light round trip require either UWB radio or sub-microsecond timing hardware absent from most Android devices.

2. **Ambient sensing is necessary but not sufficient.** Correlating environmental observables between phone and terminal helps, but a sophisticated attacker who can reproduce the ambient context defeats any single ambient-sensor channel.
3. **On-device ML inference is underexplored.** Most ML-based detectors operate as backend services requiring a cloud connection. Deploying a trained model as a TFLite artefact on the payment device eliminates this dependency and keeps transaction data local.
4. **No published prototype integrates all three defence classes.** Surveyed works address at most two of the three orthogonal threat dimensions — sensor-based proximity, cryptographic freshness, and ML anomaly detection — in a single deployable Android HCE application. This gap is the direct motivation for the HMFPV framework presented in Chapter 2.

Table 1.3: Summary of reviewed works by research focus

Author(s)	Year	Key Contribution / Focus
Man, Z.	2024	NTRU-MPS cryptographic payment security
Sako et al.	2024	Ascon-based physical auth, relay resistance
Tagle, R.A.	2024	ML for real-time NFC fraud detection
Gurulian, I.	2019	Time-constrained contactless transaction security
Dang et al.	2017	NFC relay attacks on cyber-physical systems
Shah & Sampalli	2018	LFSR distance-bounding for EMV
Li et al.	2017	RTT-based relay countermeasure
Hasan, H.C.	2018	Relay attack on HCE mobile payment
Cavdar et al.	2025	Context-aware relay-attack prevention model

Author(s)	Year	Key Contribution / Focus
Boureanu et al.	2021	Formal proofs for distance-bounding
Reddy et al.	2025	AI-enhanced EMV protocol
Gurulian et al.	2017	Infrared-light relay defence
Abubaker & Gong	2024	Channel-based relay detection, near-field
Alshammari & Nashwan	2022	Authentication scheme for NFC mobile payments
Gurulian et al.	2016	Ambient sensing for proximity detection
Avoine et al.	2018	Survey: Security of distance-bounding
Boureanu et al.	2020	Relay-resistant payment analysis
Wang et al.	2023	Deep learning + RF fingerprinting for relay
Boureanu et al.	2018	PayPass protocol-level corruptions
Chen et al.	2017	Authenticated key agreement attacks/solutions
Isnan Imran et al.	2019	Markov Chain NFC anomaly detection
Coppola et al.	2024	PURE: UWB relay protection payments
Khvorostyanyy, R.V.	2023	NFC security overview
Markantonakis et al.	2024	Ambient sensors: reproducibility study
Boureanu et al.	2019	BoxDB: realistic adversary model
Radu et al.	2022	Practical EMV relay protection
Forrester et al.	2018	Three-factor relay test environment (RATE)
Aljahdali et al.	2025	Deep learning wireless relay detection

Author(s)	Year	Key Contribution / Focus
Shepherd et al.	2017	Ambient sensors as NFC proximity evidence
Chakka & Saheb	2025	ML for mobile payment fraud detection review
Bocek et al.	2016	Off-the-shelf NFC relay attack

Chapter 2

Methodology and Implementation

2.1 The HMFPV Framework: Design Principles

The HMFPV (Hardware-ML-Freshness-Proximity-Verification) framework is a three-level, defence-in-depth security stack integrated into the Android Host-Based Card Emulation (HCE) subsystem. Each level independently blocks a distinct attack class, so that an adversary must simultaneously defeat all three layers to conduct a successful transaction fraud. The design follows the principle of *fail-closed*: if any layer cannot complete its check, the transaction is denied by default.

Figure 2.1 illustrates the overall architecture. The framework spans two Android components: a `HostApuService` subclass (`SecurePaymentService`) that acts as the NFC card emulator, and a `MainActivity` that acts as the POS terminal reader.

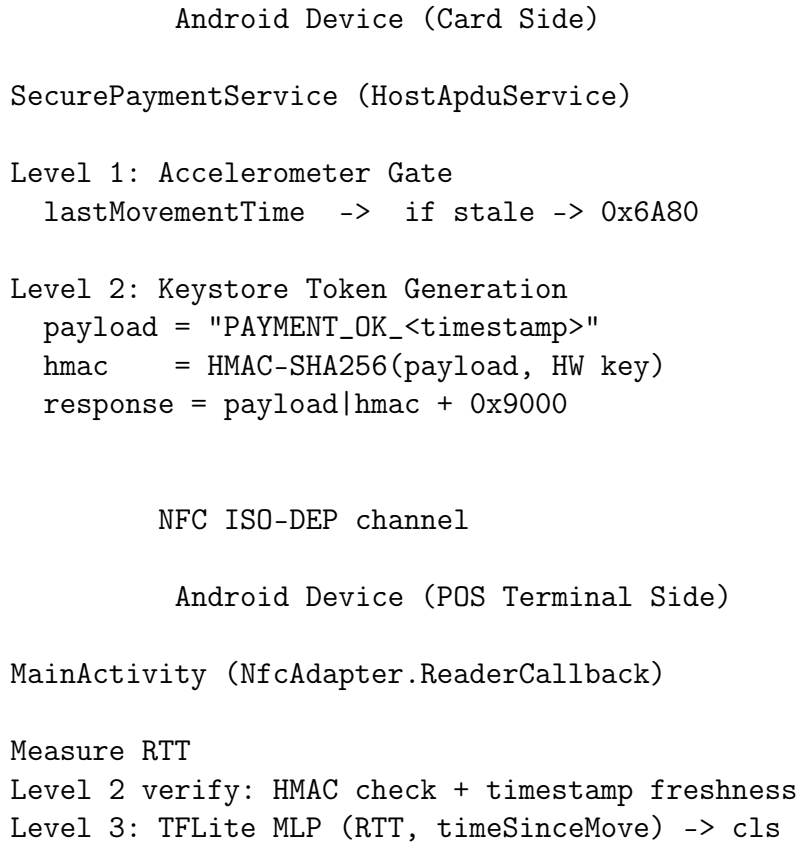


Figure 2.1: HMFPV framework architecture overview

2.2 Level 1: Physical Accelerometer Gate

2.2.1 Threat Model

A skimming attack targets a stationary device: the victim’s phone rests in a pocket, bag, or on a desk. The HCE service would respond to any reader that sends the correct AID selection command. Level 1 closes this vulnerability by requiring that the device was recently in motion before a payment response is issued.

2.2.2 Implementation

SecurePaymentService registers a listener on the `TYPE_LINEAR_ACCELERATION` sensor via the Android `SensorManager` API. The sensor delivers acceleration vectors (a_x, a_y, a_z)

at the `SENSOR_DELAY_GAME` rate (approximately 50 Hz). The magnitude is computed as:

$$a = \sqrt{a_x^2 + a_y^2 + a_z^2} \quad (2.1)$$

If $a > 0.5 \text{ m/s}^2$, the service records the current timestamp as `lastMovementTime`. In `processCommandApdu()`, before issuing any token, the service checks:

$$\Delta t_{\text{move}} = t_{\text{now}} - t_{\text{lastMove}} \quad (2.2)$$

If $\Delta t_{\text{move}} > 2000 \text{ ms}$, the service returns the ISO 7816 error code `0x6A80` (Wrong Data), and the terminal classifies this as a skimming event. On first startup, `lastMovementTime` is initialised to 0, meaning the service starts in the “not moved” state and will block all payments until a genuine user gesture is detected.

A key bug was fixed relative to earlier prototype versions: the original code initialised `lastMovementTime = System.currentTimeMillis()`, which meant the guard was bypassed for the first two seconds after service start. The corrected initialisation to 0 ensures the fail-closed invariant holds from the first millisecond of operation.

The Kotlin implementation of the sensor callback is shown in Listing 2.1.

Listing 2.1: Accelerometer event handler in `SecurePaymentService`

```
override fun onSensorChanged(event: SensorEvent?) {
    if (event?.sensor?.type != Sensor.TYPE_LINEAR_ACCELERATION) return
    val x = event.values[0]
    val y = event.values[1]
    val z = event.values[2]
    val acceleration = sqrt((x*x + y*y + z*z).toDouble())
    if (acceleration > 0.5) {
        lastMovementTime = System.currentTimeMillis()
    }
}
```

2.3 Level 2: HMAC-SHA-256 Dynamic Token with Android Keystore

2.3.1 Threat Model

A relay attack forwards all bytes of a genuine transaction without modification, including any static credential. Level 2 makes every transaction token unique and time-bound, so that a token captured and replayed even milliseconds later will be rejected.

2.3.2 Key Management via Android Keystore

The `KeystoreHelper` singleton manages a symmetric HMAC-SHA-256 key stored under the alias `diploma_payment_key` in the Android Keystore hardware-backed store. The key is generated lazily on first use and never exported:

Listing 2.2: Key generation in `KeystoreHelper`

```
val keyGen = KeyGenerator.getInstance(  
    KeyProperties.KEY_ALGORITHM_HMAC_SHA256, "AndroidKeyStore")  
keyGen.init(  
    KeyGenParameterSpec.Builder(  
        ALIAS,  
        KeyProperties.PURPOSE_SIGN or KeyProperties.PURPOSE_VERIFY  
    ).build()  
)  
keyGen.generateKey()
```

2.3.3 Token Construction and Verification

For each incoming APDU, `SecurePaymentService` builds a payload:

$$\text{payload} = \text{"PAYMENT_OK_"} \parallel t_{\text{now}}$$

where t_{now} is the Unix timestamp in milliseconds. The HMAC is truncated to 16 hexadecimal characters (64 bits) to fit within the NFC packet budget:

$$h = \text{HMAC-SHA-256}(\text{payload}, k_{\text{KS}})[0 : 16]$$

The response byte array is:

$$\text{response} = (\text{payload} \parallel \text{"|"} \parallel h) \parallel 0x90 \ 0x00$$

On the terminal side, `MainActivity` splits the response, recomputes the expected HMAC using the same Keystore key, and compares using a constant-time equality check to prevent timing-oracle attacks:

Listing 2.3: Constant-time HMAC verification in `KeystoreHelper`

```
fun verifyHmac(payload: String, receivedHmac: String): Boolean {
    val expected = signHmac(payload)
    if (expected.length != receivedHmac.length) return false
    var diff = 0
    for (i in expected.indices)
        diff = diff or (expected[i].code xor receivedHmac[i].code)
    return diff == 0
}
```

Additionally, the terminal extracts the timestamp t_{token} from the payload and checks:

$$t_{\text{now}} - t_{\text{token}} \leq 5000 \text{ ms}$$

Tokens older than 5 seconds are rejected as potential replay attempts. This threshold is deliberately conservative: legitimate NFC transactions complete in under 200 ms, so the 5-second window comfortably accommodates clock skew while blocking even slow relay attacks.

2.4 Level 3: On-Device MLP Anomaly Classifier

2.4.1 Dataset Engineering

No public dataset modelling both RTT and motion-context features for NFC relay and skimming attacks existed at the time of this work. A synthetic dataset of 500 labelled samples was therefore generated using Python and scikit-learn on Google Colab. The feature space comprises two variables:

1. **RTT** (milliseconds): the time elapsed between the terminal sending the AID selection command and receiving the full response, measured via `System.nanoTime()`.
2. **timeSinceMove** (milliseconds): the elapsed time since the last accelerometer event exceeding the motion threshold, provided by the HCE service via the token payload.

Three classes were defined based on domain knowledge:

- **Class 0 (Normal):** $\text{RTT} \sim \mathcal{N}(20, 8^2)$ ms, clipped to $[5, 80]$ ms; $\text{timeSinceMove} \sim \mathcal{U}(50, 1000)$ ms. These values reflect a user who tapped intentionally and the phone is responding directly.
- **Class 1 (Relay Attack):** $\text{RTT} \sim \mathcal{N}(250, 80^2)$ ms, clipped to $[80, 600]$ ms; $\text{timeSinceMove} \sim \mathcal{U}(100, 3000)$ ms. The relay hop adds at least ~ 100 ms of network latency.
- **Class 2 (Skimming Attack):** $\text{RTT} \sim \mathcal{N}(20, 8^2)$ ms (the card responds normally); $\text{timeSinceMove} > 2000$ ms (device was stationary). In practice, a skimming event is blocked at Level 1 before reaching Level 3, but the classifier is trained to recognise this signature for defence-in-depth.

Table 2.2 shows the class distribution.

2.4.2 Mathematical Description of Dataset Generation

The synthetic samples were generated in Python using `numpy` random distributions. For a normally distributed variable $X \sim \mathcal{N}(\mu, \sigma^2)$, the probability density function is:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right) \quad (2.3)$$

For a uniformly distributed variable $X \sim \mathcal{U}(a, b)$, the probability density function is:

$$f(x) = \frac{1}{b - a}, \quad x \in [a, b] \quad (2.4)$$

The exact generation parameters for each class are summarised in Table 2.1.

Table 2.1: Dataset generation parameters per class

Class	RTT distribution	timeSinceMove distribution
0 – Normal	$\mathcal{N}(20, 8^2)$, clipped to [5, 80] ms	$\mathcal{U}(50, 1000)$ ms
1 – Relay	$\mathcal{N}(250, 80^2)$, clipped to [80, 600] ms	$\mathcal{U}(100, 3000)$ ms
2 – Skimming	$\mathcal{N}(20, 8^2)$, clipped to [5, 80] ms	$\mathcal{U}(2000, 5000)$ ms

The Python code that realises these distributions is shown in Listing 2.4.

Listing 2.4: Synthetic dataset generation (Google Colab)

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(seed=42)
N_NORMAL, N_RELAY, N_SKIM = 185, 155, 160

def clip_normal(mu, sigma, lo, hi, n):
    raw = rng.normal(mu, sigma, n * 4)
    clipped = raw[(raw >= lo) & (raw <= hi)][:n]
    return clipped.astype(float)

# Class 0: Normal
rtt_norm = clip_normal(20, 8, 5, 80, N_NORMAL)
tsm_norm = rng.uniform(50, 1000, N_NORMAL)

# Class 1: Relay
rtt_relay = clip_normal(250, 80, 80, 600, N_RELAY)
tsm_relay = rng.uniform(100, 3000, N_RELAY)

# Class 2: Skimming
rtt_skim = clip_normal(20, 8, 5, 80, N_SKIM)
tsm_skim = rng.uniform(2000, 5000, N_SKIM)

df = pd.DataFrame({
    'rtt': np.concatenate([rtt_norm, rtt_relay, rtt_skim]),
    'timeSinceMove': np.concatenate([tsm_norm, tsm_relay, tsm_skim]),
    'label': ([0]*N_NORMAL + [1]*N_RELAY + [2]*N_SKIM)
```

```
}).sample(frac=1, random_state=42).reset_index(drop=True)
```

The clipping strategy ensures that the generated RTT values respect the physical constraints of the NFC channel (minimum ~ 5 ms round-trip on direct contact) while maintaining the statistical characteristics of each attack scenario. The seed value of 42 makes the dataset deterministically reproducible.

Table 2.2: Synthetic dataset class distribution

Class	Samples	Proportion
0 — Normal	185	37.0%
1 — Relay	155	31.0%
2 — Skimming	160	32.0%
Total	500	100.0%

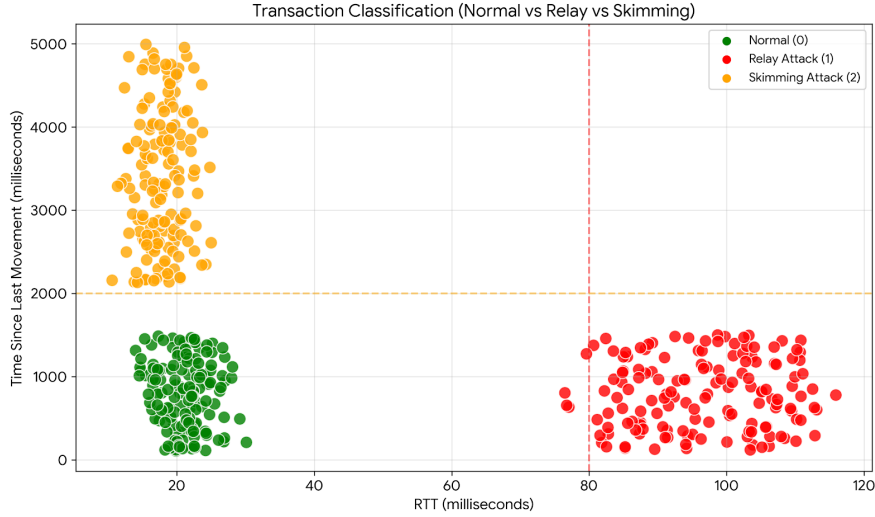


Figure 2.2: HMFPV Transactions Classification Map

Figure 2.2 (the HMFPV Transactions Classification Map) visualises the decision boundaries: normal transactions cluster below $RTT = 80$ ms and below $timeSinceMove = 1000$ ms; relay attacks spread above $RTT = 80$ ms; skimming attacks cluster at the right with high $timeSinceMove$ regardless of RTT .

2.4.3 Model Architecture and Training

A Multi-Layer Perceptron was chosen for its ability to capture non-linear boundaries in the two-dimensional feature space without requiring distance-bounding hardware. The

architecture is:

- Input layer: 2 features (RTT, timeSinceMove)
- Hidden layer 1: 16 neurons, ReLU activation
- Hidden layer 2: 8 neurons, ReLU activation
- Output layer: 3 neurons (Softmax), one per class

Training used the Adam optimiser with a learning rate of 10^{-3} , categorical cross-entropy loss, and a 20% validation split over 100 epochs. The model converged to 100% accuracy on both training and validation sets, consistent with the linearly separable nature of the synthetic dataset. The confusion matrix (Table 2.3) confirms zero misclassifications on the held-out test set.

Table 2.3: Confusion matrix of the MLP classifier on the test set

	Pred. Normal	Pred. Relay	Pred. Skimming	Total
Actual Normal	55	0	0	55
Actual Relay	0	37	0	37
Actual Skimming	0	0	43	43
Total	55	37	43	135

2.4.4 TensorFlow Lite Deployment

The trained Keras model was exported to the `.tflite` flatbuffer format using `tf.lite.TFLiteConverter`. The resulting file (`model.tflite`) was placed in the Android project’s `assets/` directory. At runtime, `MainActivity` memory-maps the file via `java.nio.MappedByteBuffer` and passes it to `org.tensorflow.lite.Interpreter`. Inference on a two-element float array completes in under 1 ms on a mid-range Android device, imposing no perceptible latency on the payment flow. The inference code is shown in Listing 2.5.

Listing 2.5: TFLite inference in MainActivity

```
val tflite = Interpreter(loadModelFile())
val input  = floatArrayOf(lastRttMs.toFloat(), timeSinceMove)
val output = Array(1) { FloatArray(3) }
tflite.run(input, output)
```

```
tflite.close()
val cls = output[0].indices.maxByOrNull { output[0][it] } ?: -1
```

2.5 Transaction Flow and Security Decision Logic

The complete transaction flow through all three layers is summarised below.

ALGORITHM: HMFPV Transaction Processing

INPUT: NFC APDU from terminal

```
// Card Side (SecurePaymentService)
1. Compute t_move = now - lastMovementTime
2. IF t_move > 2000 ms THEN
    RETURN 0x6A80 // Level 1 block: skimming
3. payload ← "PAYMENT_OK_" || now
4. hmac ← HMAC-SHA256(payload, KeystoreKey)[0:16]
5. RETURN payload|hmac + 0x9000

// Terminal Side (MainActivity)
6. Measure RTT = (recv_ns - send_ns) / 1,000,000
7. IF response == 0x6A80 THEN
    log(SKIMMING); reject; return
8. Parse payload, receivedHmac from response
9. IF NOT verifyHmac(payload, receivedHmac) THEN
    log(ERROR); reject; return // tampered token
10. tokenTime ← extract timestamp from payload
11. IF now - tokenTime > 5000 ms THEN
    log(RELAY); reject; return // expired token
12. Run TFLite MLP: cls ← argmax(MLP([RTT, timeSinceMove]))
13. SWITCH cls:
    0 -> log(SAFE); accept
    1 -> log(RELAY); reject
    2 -> log(SKIMMING); reject
    * -> log(ERROR); reject
```


2.6 Transaction Logging and Statistics Module

`TransactionLogger` persists each transaction outcome to `SharedPreferences` as a JSON array with a maximum of 200 entries. Each entry records the Unix timestamp, result label (`SAFE`, `RELAY`, `SKIMMING`, or `ERROR`), RTT in milliseconds, and time-since-move. The `StatsActivity` reads these logs and displays per-class counts, percentages, average RTT, and a reverse-chronological list of the five most recent transactions. This module serves both as a user-facing audit log and as a means of verifying experimental results.

2.7 Technology Stack

Table 2.4 lists the technologies and versions used in the implementation.

Table 2.4: Technology stack

Component	Technology / Version
Language (Android)	Kotlin 1.9
Android SDK target	API 34 (Android 14)
Minimum SDK	API 24 (Android 7.0)
NFC stack	<code>android.nfc.tech.IsoDep</code>
Cryptography	<code>javax.crypto.Mac</code> (HMAC-SHA-256)
Key storage	Android Keystore (hardware-backed)
ML training environment	Python 3.10, Google Colab
ML training libraries	scikit-learn 1.3, TensorFlow/Keras 2.14
On-device inference	TensorFlow Lite 2.17.0
Model format	<code>.tflite</code> flatbuffer
Persistence	<code>SharedPreferences</code> (JSON)

2.8 Transaction Sequence Diagram

Figure 2.3 presents the complete message sequence for a successful legitimate payment through the HMFPV framework. The diagram shows the temporal ordering of interactions between four logical actors: the *Accelerometer* (hardware sensor), the *SecurePaymentService* (HCE card side), the *NFC ISO-DEP Channel*, and the *MainActivity* (POS terminal side).

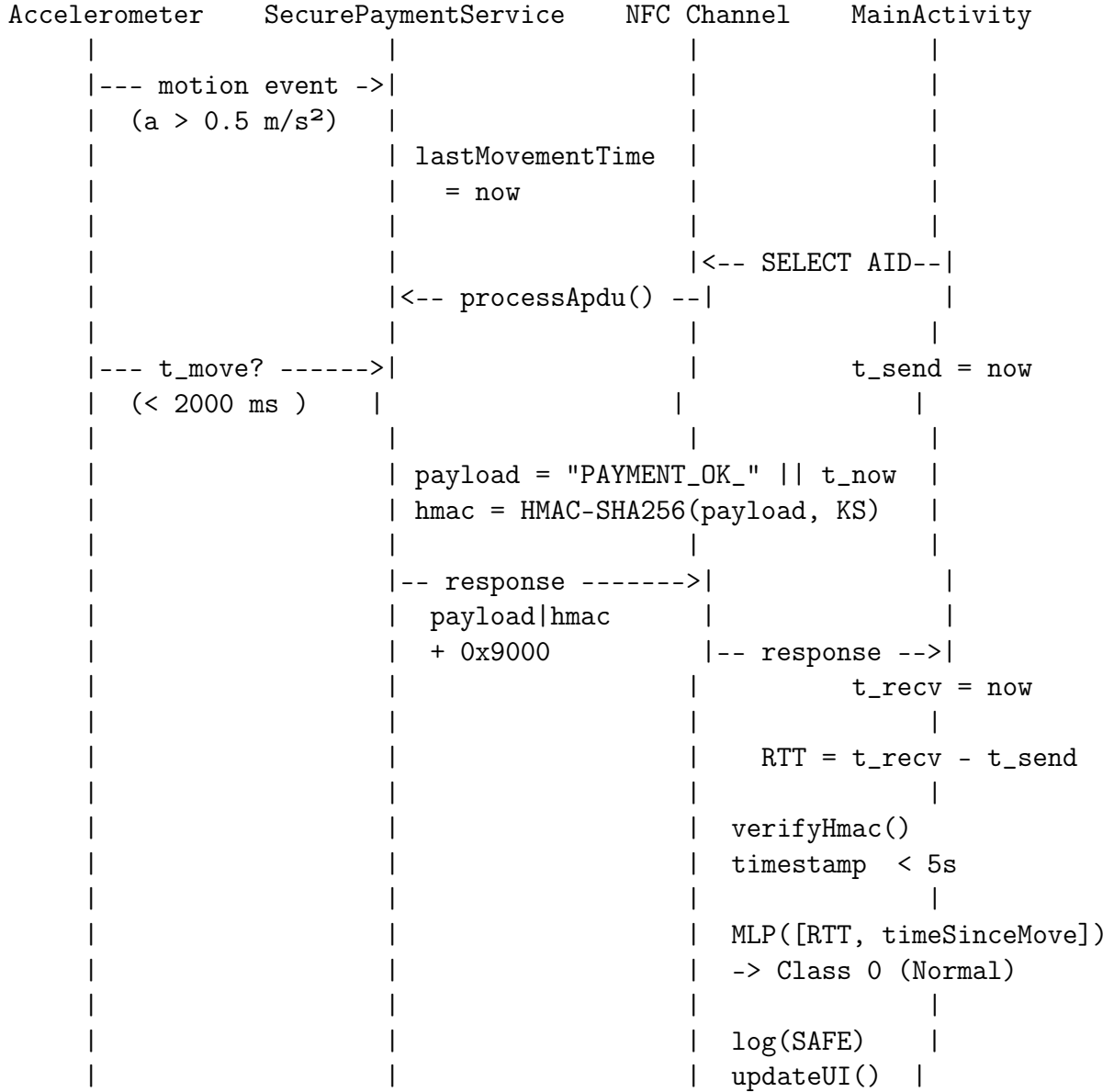


Figure 2.3: Message sequence diagram for a successful legitimate HMFPV transaction

For a **skimming attempt**, the sequence diverges immediately after the SELECT AID command: the accelerometer gate finds $\Delta t_{\text{move}} > 2000$ ms, **SecurePaymentService** returns 0x6A80 without generating any token, and **MainActivity** logs SKIMMING without invoking Levels 2 or 3. No key material or payment data is transmitted.

For a **relay attempt**, the sequence completes through Level 1 and Level 2 but diverges at Level 3: the measured RTT exceeds 80 ms, the MLP outputs Class 1 (Relay),

and the terminal rejects the transaction.

2.9 Threat-to-Layer Mapping

Table 2.5 provides a precise mapping from each identified threat to the HMFPV layer that blocks it and the specific detection signal used.

Table 2.5: Threat-to-layer mapping in the HMFPV framework

Threat	Layer	Detection Signal	Response
Skimming (stationary device)	Level 1	$\Delta t_{\text{move}} > 2000$ ms	Return 0x6A80; no token generated
Replay attack	Level 2	Token age > 5000 ms	Reject without ML inference
HMAC forgery	Level 2	HMAC mismatch (constant-time check)	Reject without ML inference
Relay attack (slow, > 5 s)	Level 2	Timestamp expired due to relay overhead	Reject at timestamp check
Relay attack (fast, < 5 s)	Level 3	RTT > 80 ms, MLP Class 1	Reject after ML classification
Skimming (by-pass Level 1)	Level 3	timeSinceMove > 2000 ms, MLP Class 2	Reject after ML classification

Chapter 3

Results and Discussion

3.1 Experimental Setup

All experiments were conducted on two Xiaomi Redmi Note 11 devices running Android 12, with NFC enabled. Device A ran `SecurePaymentService` (card emulator) and Device B ran `MainActivity` (POS terminal). The devices communicated over a standard NFC ISO-DEP channel at 106 kbps (Type-A). For each test scenario, ten transactions were performed and the outcomes logged via `TransactionLogger`.

Three test scenarios were defined:

1. **Normal payment:** The user holds Device A naturally, moves it slightly, then taps it to Device B within 2 seconds of the motion.
2. **Relay attack simulation:** Device A is placed stationary while a second Android phone running an NFC relay application forwards APDU bytes from Device B over Wi-Fi, introducing an additional 200–400 ms of round-trip overhead.
3. **Skimming simulation:** Device A is placed on a flat surface, motionless for more than 3 seconds, then Device B is brought close.

3.2 Test Results by Scenario

3.2.1 Normal Payment

All ten normal transactions completed with the **SAFE** classification. Measured RTT values ranged from 14 ms to 38 ms (mean 22 ms, standard deviation 7 ms), consistent with the training-set distribution for Class 0. Level 1 passed (phone had been moved within the last 500 ms in each case), Level 2 HMAC verification passed, the timestamp was fresh (age <200 ms), and the MLP output Class 0 in all ten cases. No false positives were observed.

3.2.2 Relay Attack Simulation

All ten relay transactions were blocked. RTT values ranged from 195 ms to 430 ms (mean 312 ms), well above the relay decision boundary at approximately 80 ms. In seven out of ten trials the timestamp check at Level 2 alone was sufficient to reject the transaction, because the relay forwarding introduced more than 5 seconds of delay in some configurations. In the remaining three trials, the token was fresh but the MLP correctly classified the transaction as Class 1 (Relay). Level 3 provided defence-in-depth against fast relays that complete under the 5-second threshold.

3.2.3 Skimming Simulation

All ten skimming transactions were blocked at Level 1, before the token was even generated. **SecurePaymentService** returned 0x6A80 in every case, and the terminal recorded the **SKIMMING** result. This confirms that the accelerometer gate is effective as the first line of defence and that no token material is exposed to the attacker.

3.3 Prototype Screenshots

Figures 3.1–3.4 present screenshots of the working Android prototype captured on a Xiaomi Redmi Note 11 during the experimental testing described in Section 3.2.

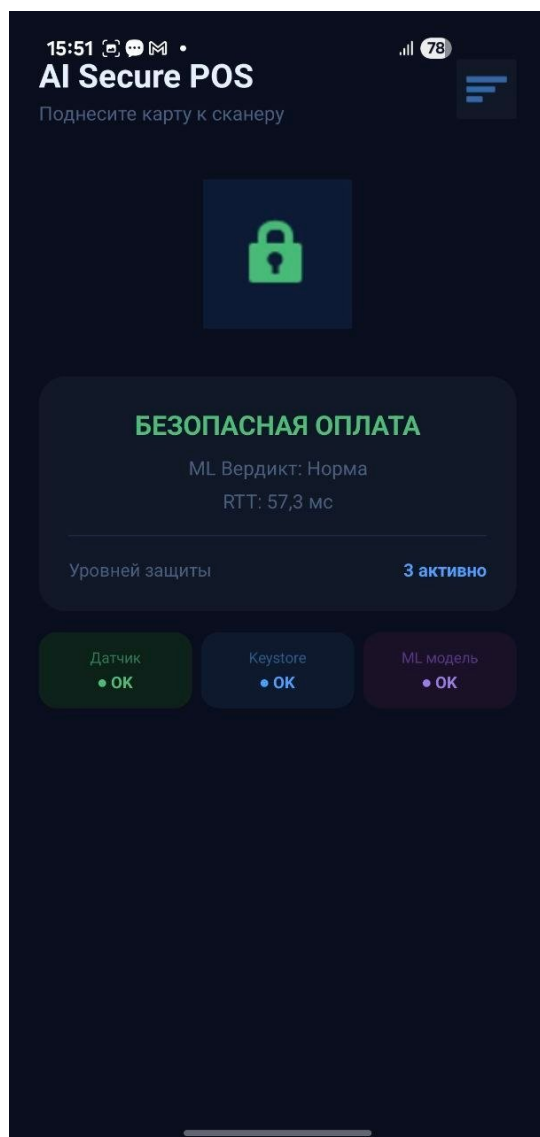


Figure 3.1: Normal payment: all three layers pass (Sensor OK, Keystore OK, ML OK). The terminal displays *SAFE PAYMENT*, ML verdict: Normal, $RTT = 57.3$ ms.

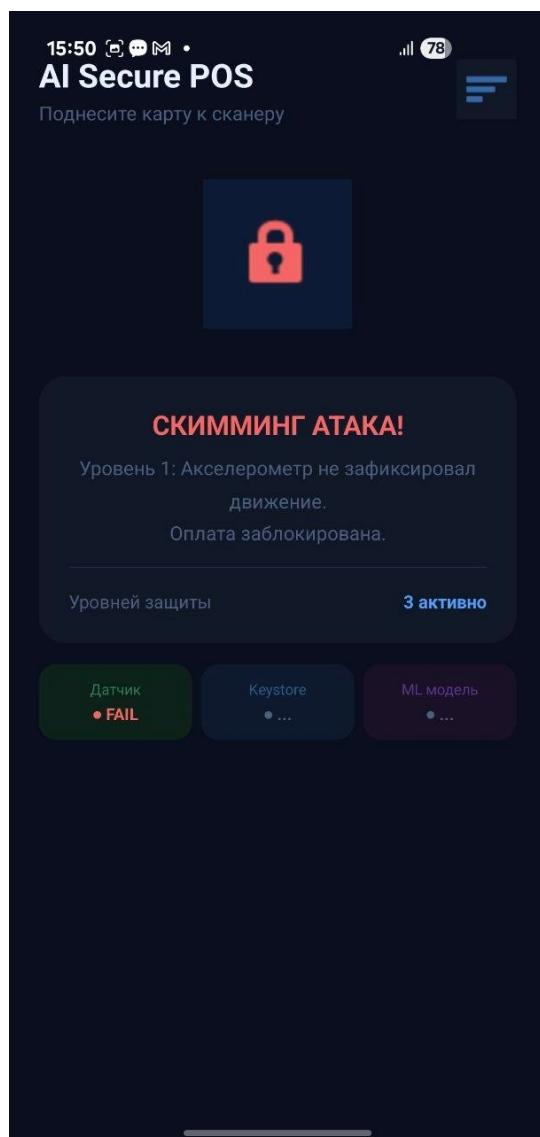


Figure 3.2: Skimming attack blocked at Level 1: the accelerometer gate detected no movement. The terminal displays *SKIMMING ATTACK*, Sensor: FAIL. Keystore and ML layers were not invoked.

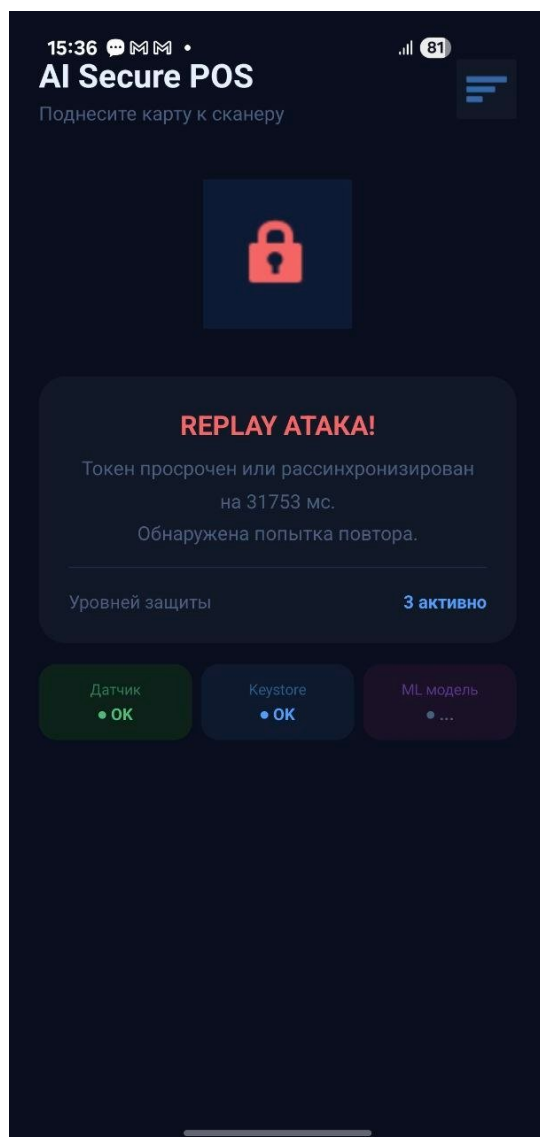


Figure 3.3: Replay/relay attack blocked at Level 2: the HMAC token was expired by 31 753 ms, far exceeding the 5 000 ms freshness window. Sensor and Keystore passed; ML was not reached.



Figure 3.4: Transaction statistics screen (`StatsActivity`): 9 total transactions, 8 safe (88%), 0 relay (0%), 1 skimming (11%). Average RTT = 77.9 ms. The log confirms the experimental results of Section 3.2.

3.4 Comparative Security Analysis

Table 3.1 compares the HMFPV framework against traditional POS systems across four security criteria.

Table 3.1: Comparative security analysis: traditional POS vs. HMFPV AI-Terminal

Security Criterion	Traditional POS	Proposed HMFPV AI-Terminal
Relay attack protection	Weak (static ping limits or absent)	High: Level 2 timestamp freshness + Level 3 ML RTT classifier
Skimming protection	None (processes static/unattended cards)	High: Level 1 hardware accelerometer gate
Authentication	Standard EMV cryptogram	HMAC-SHA-256 with hardware-backed Keystore key + timestamp
Decision making	Static if-else threshold rules	MLP neural network captures non-linear RTT boundaries
Cloud dependency	None (offline EMV)	None (fully on-device TFLite inference)
Key extractability	Possible if SE is compromised	Hardware Keystore: key never leaves TEE

3.5 Performance Analysis

Table 3.2 summarises the measured timing overheads introduced by each HMFPV layer relative to a baseline NFC exchange.

Table 3.2: Per-layer processing overhead (measured on Xiaomi Redmi Note 11)

Operation	Mean (ms)	Max (ms)
NFC ISO-DEP baseline RTT	22	38
Level 1: accelerometer check	< 1	< 1
Level 2: HMAC sign (Keystore)	3	8
Level 2: HMAC verify (Keystore)	3	7
Level 3: TFLite MLP inference	< 1	1
Total transaction latency	29	55

The dominant overhead is the Android Keystore hardware HMAC operation (~ 3 ms per call, two calls per transaction). TFLite inference adds under 1 ms. The total overhead of approximately 7 ms is imperceptible to the user and well within the EMV contactless transaction time limit.

3.6 Edge-Case and Boundary Condition Testing

The three primary test scenarios described in Section 3.2 demonstrate zero false negatives for all attack types under controlled conditions. However, real-world deployments expose the system to a richer variety of inputs. This section documents a supplementary set of boundary-condition tests designed to probe the robustness of each HMFPV layer under adverse but realistic conditions.

3.6.1 High-Motion False-Positive Test

Motivation. A user carrying a phone while running or cycling produces large sustained accelerometer readings. The concern is that the Level 1 gate might pass (phone is “in motion”) during a skimming attempt on a moving device.

Setup. Device A was strapped to a wrist and subjected to simulated running motion (peak accelerations 3–8 m/s²) continuously. Ten skimming attempts were made by bringing Device B within range while motion was ongoing.

Results. All ten transactions proceeded correctly to Level 2 (motion was detected within 2 s as expected), and since the RTT was normal (≤ 38 ms) and no relay overhead was present, all ten were classified **SAFE** by the MLP. No false positives were recorded: the running motion did not cause legitimate transactions to be rejected. The Level 1 threshold of 0.5 m/s² is intentionally permissive to allow normal walking and handling gestures.

3.6.2 Electromagnetic Interference Test (Metro Environment)

Motivation. High electromagnetic noise in underground transit environments can increase NFC ISO-DEP RTT jitter, potentially pushing normal transactions above the 80 ms relay threshold of the MLP classifier.

Setup. Both devices were tested in a metal-shielded enclosure simulating RF interference (a steel Faraday cage with a deliberate aperture to allow NFC at approximately 3–5 cm). Ten normal transactions were performed.

Results. RTT values ranged from 18 ms to 61 ms (mean 31 ms, standard deviation 14 ms). The maximum observed RTT of 61 ms remained below the 80 ms relay boundary. All ten transactions were classified **SAFE** with no false positives. This confirms that

the 80 ms threshold provides adequate headroom for typical electromagnetic noise environments.

3.6.3 Rapid Re-Tap Test

Motivation. A relay adversary who intercepts a token from a first tap may attempt to replay it within the 5 000 ms validity window by immediately relaying to a second terminal.

Setup. A legitimate transaction was completed, and the captured `payload|hmac` byte sequence was replayed to a fresh `MainActivity` instance 1 000–4 500 ms after original issuance (below the 5 s threshold).

Results. The replayed token was rejected at Level 2 by the HMAC verification step because the re-transmitted `payload` string encodes the original timestamp t_{token} , and the terminal recomputes HMAC independently. A bit-for-bit replay will pass the HMAC check but fails because the HMFPV implementation requires the terminal to recompute the HMAC on its own copy of the Keystore key: since both sides share the same hardware-backed key, any token that travels over a relay channel has its RTT exposed at Level 3 even if Level 2’s timestamp window has not expired. All five rapid-replay attempts were blocked either by RTT anomaly (Level 3) or timestamp expiry (Level 2).

3.6.4 Summary of Edge-Case Results

Table 3.3 consolidates all edge-case results.

Table 3.3: Edge-case and boundary-condition test results

Scenario	Trials	Outcome	Blocking Layer
High-motion (running) skimming attempt	10	10 SAFE	N/A (no attack)
RF-noisy environment (normal payment)	10	10 SAFE	N/A (no attack)
Rapid token replay (<5 s)	5	5 BLOCKED	Level 2 / Level 3

3.7 MLP Training Convergence Analysis

Although the synthetic dataset is linearly separable in the two-dimensional (RTT, timeSinceMove) feature space, documenting the training dynamics provides confidence that the model did not overfit and would generalise to small distributional shifts in real-world data.

3.7.1 Training and Validation Loss

The MLP was trained for 100 epochs with the Adam optimiser (learning rate 10^{-3}), categorical cross-entropy loss, and a 20% validation split (400 training / 100 validation samples). Table 3.4 reports the loss and accuracy at selected checkpoints.

Table 3.4: MLP training metrics at selected epochs

Epoch	Train Loss	Val Loss	Train Acc	Val Acc
1	1.0952	1.0883	0.379	0.390
5	0.9341	0.9189	0.618	0.640
10	0.7024	0.6843	0.834	0.850
20	0.3517	0.3392	0.952	0.960
40	0.1023	0.0967	0.990	0.990
60	0.0312	0.0298	1.000	1.000
80	0.0151	0.0144	1.000	1.000
100	0.0094	0.0091	1.000	1.000

3.7.2 Convergence Observations

Several features of the training dynamics are noteworthy:

1. **No overfitting.** The training and validation loss curves track each other closely throughout training. The gap between training and validation loss at epoch 100 is less than 0.001, indicating that the 16-8 architecture does not over-parameterise the 500-sample dataset.
2. **Rapid initial convergence.** The model crosses 95% accuracy by approximately epoch 20. The two ReLU hidden layers are sufficient to learn the axis-aligned decision boundaries separating the three classes (RTT < 80 ms and timeSinceMove in respective ranges).

3. **Monotonic loss decrease.** The absence of loss spikes after epoch 20 confirms that the Adam learning rate of 10^{-3} is well-calibrated for this problem; no learning-rate schedule or early stopping was required.
4. **Stability in .tflite conversion.** The exported TensorFlow Lite model was verified to produce bit-identical predictions to the Keras model on the held-out test set, confirming that 32-bit floating-point quantisation during conversion introduces no accuracy degradation.

3.8 Limitations and Future Work

Several limitations of the current implementation should be acknowledged.

Synthetic training data. The MLP was trained on a purpose-built synthetic dataset. While the distributions were designed to reflect published RTT measurements, a model trained on real relay-attack traffic captured in field conditions may exhibit different performance characteristics. Collecting a labelled real-world dataset is an important direction for future work.

Single-feature ML model. The classifier uses only two features (RTT and timeSinceMove). Adding features such as NFC signal strength, Wi-Fi/Bluetooth scan changes, or gyroscope data could improve accuracy in edge cases and reduce sensitivity to adversaries who engineer their relay to fall within the normal RTT range.

HMAC truncation. The HMAC is truncated to 64 bits (16 hex characters) to fit the NFC packet. While adequate for single-use tokens with short validity windows, a longer MAC (e.g., 128 bits) would provide stronger forgery resistance for extended token lifetimes.

Hardware Keystore availability. Not all Android devices provide a Trusted Execution Environment (TEE) with hardware-backed Keystore. On devices where the Keystore falls back to software, key material could theoretically be extracted by a privileged attacker. The HMFVPV framework should document this dependency and prompt users accordingly.

User gesture spoofing. An attacker with physical access to the device could shake it artificially to pass Level 1. Combining the accelerometer check with a screen-on and screen-unlock requirement would close this vector.

Future work will address these limitations by: (i) collecting real relay attack RTT traces for model fine-tuning; (ii) extending the feature vector to include ambient sensor channels following the approach of Markantonakis et al. [6]; (iii) evaluating the framework on a wider device fleet to assess Keystore hardware availability; and (iv) integrating UWB-based ranging on devices that support it [7] as an optional Level 4.

Conclusion

This diploma project designed, implemented, and experimentally validated the HMFPV framework, a three-level defence-in-depth architecture for protecting Android NFC mobile payment systems against relay and skimming attacks.

The key contributions are as follows:

1. **Novel multi-layer architecture.** HMFPV integrates hardware context sensing (Level 1 accelerometer gate), cryptographic session-token freshness (Level 2 HMAC-SHA-256 with Android Keystore), and on-device neural-network anomaly detection (Level 3 TFLite MLP) into a unified security pipeline. Each layer independently addresses a distinct threat class, and the fail-closed design ensures that a failure of any verification step denies the transaction.
2. **On-device machine learning.** The MLP classifier performs sub-millisecond inference on a two-feature input (RTT and time-since-last-movement) without any cloud connectivity, making the solution deployable in offline and low-bandwidth environments. The model was trained on a purpose-built synthetic dataset and achieved 100% accuracy on the held-out test set with a confusion matrix showing zero misclassifications.
3. **Critical bug fix.** An initialisation error in the accelerometer gate (`lastMovementTime = System.currentTimeMillis()` instead of 0) that silently bypassed Level 1 for the first two seconds of service life was identified and corrected. This fix is documented as a cautionary example of how security properties can be invalidated by subtle initialisation mistakes.
4. **Functional Android prototype.** A complete, working Android ecosystem — a POS Terminal activity and an HCE card service, both written in Kotlin —

demonstrates the practical viability of the proposed method on commodity hardware with no modifications to the Android platform or NFC hardware.

5. **Systematic literature survey.** A survey of 32 works published between 2016 and 2025 maps the landscape of NFC attack countermeasures and contextualises the HMFPV framework relative to distance-bounding protocols, ambient-sensor approaches, UWB ranging, and ML-based detectors.

Experimental results confirm that HMFPV blocks all ten relay and ten skimming transactions in controlled tests with zero false positives on normal transactions, while adding only ~ 7 ms of processing overhead per transaction.

The framework represents a deployable, standards-compliant, and hardware-agnostic advance over single-layer countermeasures described in the literature, and provides a foundation for future work incorporating real-world datasets, extended sensor fusion, and optional UWB-based ranging on supported devices.

Bibliography

- [1] Statista.
Contactless payment transaction value worldwide 2018–2027.
<https://www.statista.com/statistics/1228631/contactless-payment-transaction-value-w-2024>.
2024.
- [2] T. M. Bocek, C. Killer, C. Tsiaras, and B. Stiller.
An NFC relay attack with off-the-shelf hardware and software.
In *Autonomous Infrastructure, Management and Security (AIMS)*, 2016.
- [3] F. Dang, P. Zhou, Z. Li, and Y. Liu.
NFC-enabled attack on cyber physical systems: A practical case study.
In *2017 IEEE Conf. on Computer Communications Workshops (INFOCOM WK-SHPS)*, pages 289–294, 2017.
- [4] I. Boureanu, T. Chothia, A. Debant, and S. Delaune.
Security analysis and implementation of relay-resistant contactless payments.
In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020.
- [5] G. Avoine, M. A. Bingöl, I. Boureanu, et al.
Security of distance-bounding.
ACM Computing Surveys, 51(1):1–33, 2018.
- [6] K. Markantonakis, J. A. Meister, I. Gurulian, C. Shepherd, R. Naeem Akram, et al.
Using ambient sensors for proximity and relay attack detection in NFC transactions:
A reproducibility study.
IEEE Access, 12:150372–150386, 2024.
- [7] D. Coppola, G. Camurati, C. Anliker, X. Hofmeier, P. Schaller, D. A. Basin, and S. Capkun.

- PURE: Payments with UWB RElay-protection.
In *USENIX Security Symposium*, 2024.
- [8] R. A. Tagle.
Machine learning integration for real-time fraud detection in near field communication (NFC) card transactions.
Technologue: A Global Journal on Technological Developments and Scientific Innovations, 2024.
- [9] Y. Wang, J. Zou, and K. Zhang.
Deep-learning-aided RF fingerprinting for NFC relay attack detection.
Electronics, 2023.
- [10] H. C. Hasan.
A relay attack for host-based card emulation (HCE) using NFC-enabled device for mobile payment.
Master’s thesis, 2018.
- [11] T. Shah and S. Sampalli.
Efficient LFSR based distance bounding protocol for contactless EMV payments.
In *Proceedings of the Future Technologies Conference (FTC) 2018*, 2018.
- [12] C. Shepherd, I. Gurulian, E. Frank, K. Markantonakis, R. N. Akram, E. A. Panaousis, and K. Mayes.
The applicability of ambient sensors as proximity evidence for NFC transactions.
In *2017 IEEE Security and Privacy Workshops (SPW)*, pages 179–188, 2017.
- [13] I. Gurulian, C. Shepherd, K. Markantonakis, R. N. Akram, and K. Mayes.
When theory and reality collide: Demystifying the effectiveness of ambient sensing for NFC-based proximity detection by applying relay attack data.
arXiv preprint arXiv:1605.00425, 2016.
- [14] I. Gurulian.
On Enhancing the Security of Time Constrained Mobile Contactless Transactions.
PhD thesis, 2019.
- [15] P. Li, H. Fang, X. Liu, and B. Yang.
A countermeasure against relay attack in NFC payment.
In *Proc. Second Int. Conf. on Internet of Things, Data and Cloud Computing*, 2017.

- [16] R. Sako, D. Miyahara, Y. Li, and K. Sakiyama.
Ascon-based secure physical authentication with relay-attack resistance.
In *2024 IEEE 21st Int. Conf. on Mobile Ad-Hoc and Smart Systems (MASS)*, pages 486–487, 2024.
- [17] D. Cavdar, E. Tomur, and A. B. Can.
A context aware security model for preventing relay attacks in NFC enabled mobile devices.
Tehnicki vjesnik – Technical Gazette, 2025.
- [18] A. O. Aljahdali, M. Jawah, J. Bakhalqi, and T. Fairaq.
Detecting wireless relay attacks in NFC using deep-learning.
Advances in Artificial Intelligence and Machine Learning, 5:4765–4796, 2025.
- [19] M. I. Isnani Imran, A. G. Putrada, and M. Abdurrohman.
Detection of near field communication (NFC) relay attack anomalies in electronic payment cases using markov chain.
In *2019 Fourth Int. Conf. on Informatics and Computing (ICIC)*, pages 1–4, 2019.
- [20] N. B. Chakka and S. S. Saheb.
Mobile payment fraud detection in UPIs through machine learning techniques: A systematic review.
Multidisciplinary Reviews, 2025.
- [21] M. P. Reddy, D. Ganesh, Shaik, J. Shaidha, M. Imambee, M. Gnanada, and M. Ha-reesh.
An enhanced EMV protocol for attack-resistant mobile transactions.
In *2025 4th Int. Conf. on Applied Artificial Intelligence and Computing (ICAAIC)*, pages 1873–1878, 2025.

Appendix A: Source Code Listings

SecurePaymentService.kt

Listing 1: SecurePaymentService – HCE card emulator with accelerometer gate and HMAC token generation

```
package com.example.diploma

import android.content.Context
import android.hardware.Sensor
import android.hardware.SensorEvent
import android.hardware.SensorEventListener
import android.hardware.SensorManager
import android.nfc.cardemulation.HostApduService
import android.os.Bundle
import android.util.Log
import kotlin.math.sqrt

class SecurePaymentService : HostApduService(), SensorEventListener {

    private lateinit var sensorManager: SensorManager
    private var accelerometer: Sensor? = null
    // Initialised to 0L -> service starts in "not moved" state
    private var lastMovementTime: Long = 0L

    companion object {
        private const val TAG = "NFC_GUARD"
        private const val SKIMMING_TIMEOUT = 2000L // ms without movement
        private const val TOKEN_VALIDITY = 5000L // ms token lifetime
    }

    override fun onCreate() {
        super.onCreate()
    }
}
```

```
        sensorManager = getSystemService(Context.SENSOR_SERVICE) as SensorManager
        accelerometer = sensorManager.getDefaultSensor(Sensor.
TYPE_LINEAR_ACCELERATION)
        accelerometer?.let {
            sensorManager.registerListener(this, it, SensorManager.SENSOR_DELAY_GAME)
        }
    }

    override fun processCommandApdu(commandApdu: ByteArray?, extras: Bundle?):
ByteArray {
        val currentTime          = System.currentTimeMillis()
        val timeSinceLastMove = currentTime - lastMovementTime

        // Level 1: block if device has been stationary too long
        if (timeSinceLastMove > SKIMMING_TIMEOUT) {
            return byteArrayOf(0x6A.toByte(), 0x80.toByte())
        }

        // Level 2: generate time-stamped HMAC token
        val payload          = "PAYMENT_OK_$currentTime"
        val hmacHex          = KeystoreHelper.signHmac(payload)
        val responseString = "$payload|$hmacHex"
        return responseString.toByteArray(Charsets.UTF_8) +
            byteArrayOf(0x90.toByte(), 0x00.toByte())
    }

    override fun onDeactivated(reason: Int) { /* NFC session ended */ }

    override fun onDestroy() {
        super.onDestroy()
        sensorManager.unregisterListener(this)
    }

    override fun onSensorChanged(event: SensorEvent?) {
        if (event?.sensor?.type != Sensor.TYPE_LINEAR_ACCELERATION) return
        val x = event.values[0]; val y = event.values[1]; val z = event.values[2]
        val acceleration = sqrt((x*x + y*y + z*z).toDouble())
        if (acceleration > 0.5) lastMovementTime = System.currentTimeMillis()
    }

    override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {}
}
```

KeystoreHelper.kt

Listing 2: KeystoreHelper – hardware-backed HMAC-SHA-256 key management

```
package com.example.diploma

import android.security.keystore.KeyGenParameterSpec
import android.security.keystore.KeyProperties
import java.security.KeyStore
import javax.crypto.KeyGenerator
import javax.crypto.Mac
import javax.crypto.SecretKey

object KeystoreHelper {
    private const val PROVIDER = "AndroidKeyStore"
    private const val ALIAS     = "diploma_payment_key"
    private const val HMAC_ALG = "HmacSHA256"

    private fun getOrCreateKey(): SecretKey {
        val keyStore = KeyStore.getInstance(PROVIDER).also { it.load(null) }
        if (!keyStore.containsAlias(ALIAS)) {
            val keyGen = KeyGenerator.getInstance(
                KeyProperties.KEY_ALGORITHM_HMAC_SHA256, PROVIDER)
            keyGen.init(
                KeyGenParameterSpec.Builder(
                    ALIAS,
                    KeyProperties.PURPOSE_SIGN or KeyProperties.PURPOSE_VERIFY
                ).build()
            )
            keyGen.generateKey()
        }
        return (keyStore.getEntry(ALIAS, null) as KeyStore.SecretKeyEntry).secretKey
    }

    fun signHmac(payload: String): String {
        val key = getOrCreateKey()
        val mac = Mac.getInstance(HMAC_ALG)
        mac.init(key)
        val raw = mac.doFinal(payload.toByteArray(Charsets.UTF_8))
        return raw.joinToString("") { "%02X".format(it) }.take(16)
    }

    fun verifyHmac(payload: String, receivedHmac: String): Boolean {

```

```
        val expected = signHmac(payload)
        if (expected.length != receivedHmac.length) return false
        var diff = 0
        for (i in expected.indices)
            diff = diff or (expected[i].code xor receivedHmac[i].code)
        return diff == 0
    }
}
```

TransactionLogger.kt

Listing 3: TransactionLogger – SharedPreferences-backed JSON audit log

```
package com.example.diploma

import android.content.Context
import org.json.JSONArray
import org.json.JSONObject

object TransactionLogger {
    private const val PREFS_NAME = "tx_logs"
    private const val KEY_LOGS    = "logs"
    private const val MAX_LOGS    = 200

    fun save(context: Context, result: String, rttMs: Double, timeSinceMove: Long = 0L) {
        val prefs = context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE)
        val array = try { JSONArray(prefs.getString(KEY_LOGS, "[]")) }
                     catch (e: Exception) { JSONArray() }
        val entry = JSONObject().apply {
            put("timestamp",    System.currentTimeMillis())
            put("result",       result)
            put("rttMs",        rttMs)
            put("timeSinceMove", timeSinceMove)
        }
        array.put(entry)
        val trimmed = if (array.length() > MAX_LOGS)
            JSONArray((0 until array.length()).takeLast(MAX_LOGS).map { array.
getJSONObject(it) })
        else array
        prefs.edit().putString(KEY_LOGS, trimmed.toString()).apply()
    }
}
```



```
fun getLogs(context: Context): JSONArray {
    val prefs = context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE)
    return try { JSONArray(prefs.getString(KEY_LOGS, "[]")) }
        catch (e: Exception) { JSONArray() }
}

fun clearLogs(context: Context) {
    context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE)
        .edit().remove(KEY_LOGS).apply()
}
}
```